# Exploratory Testing for Charter IV

## Session Participants

Kevin He 261167637 — kevin.he2@mail.mcgill.ca (Session lead)

Nathan Audegond 261171251 — nathan.audegond@mail.mcgill.ca (Session participant)

Eric Deng 261138815 — eric.deng@mail.mcgill.ca (Session participant)

## Section 1: Session Summary

### 1.1: Session Charter

Identify capabilities and areas of potential instability of the "rest api todo list manager". Identify documented and undocumented "rest api todo list manager" capabilities. For each capability, create a script or small program to demonstrate the capability. Exercise each capability identified with data typical to the intended use of the application.

Session focus: Explore todos, projects and categories together (links, deletes, XML relationship payloads) to find orphaned or inconsistent data.

### 1.2: Version

Release version 1.5.5 of the Todo List Manager, running with profile version=3 (default data).

### 1.3: Set Up

macOS 26 with Java 25. Requests were issued with curl from the session script so that every request can be replayed exactly; responses were captured in the session log. No Postman collection was used.

### 1.4: Build

Download from https://github.com/eviltester/thingifier/releases/tag/1.5.5 — runTodoManagerRestAPI-1.5.5.jar, started with: java -jar runTodoManagerRestAPI-1.5.5.jar (API and documentation at http://localhost:4567 and http://localhost:4567/docs).

## Section 2: Session Notes

### 2.1: Session Date and Time

October 14, 2026. Duration: 45 minutes (timeboxed). Requests are numbered by step in Table 2 in the exact order they were run; the raw log preserves that order (curl logs do not carry per-request wall-clock times).

### 2.2: Exploration Notes

Table 1: Legend for request colours

| Legend | |
|---|---|
| | As Expected |
| | Bug |
| | To Explore |

Table 2: Session Notes taken during Exploratory Testing

| Time | Endpoint Tested | Response | Reference File |
|---|---|---|---|
| Setup | **Run the application through the jar file**<br>java -jar runTodoManagerRestAPI-1.5.5.jar | Application available at http://localhost:4567; default (profile version=3) data present. | runTodoManagerRestAPI-1.5.5.jar (course-provided; not included in this package) |
| Step 1 | **Test API POST request**<br>SETUP: POST /todos, POST /projects, POST /categories (one object each)<br>Documented? DOCUMENTED<br>Expected: 201 for each create | **Status 201: Created**<br>Observed: HTTP 201 — Created todo=408, project=207, category=224<br>Result: **As Expected**<br>Side note: Setup objects: todo=408, project=207, category=224 (ids from the log header; high ids reflect earlier sessions on the same server) | logs/exploratory_session4_interoperability.log exploratory/session4_interoperability.sh |
| Step 2 | **Test API POST request**<br>/projects/207/tasks body: {"id":"408"}<br>Documented? DOCUMENTED<br>Expected: 201: link todo 408 to project 207 | **Status 201: Created**<br>Observed: HTTP 201 — (empty response body)<br>Result: **As Expected**<br>Side note: Empty response body | logs/exploratory_session4_interoperability.log exploratory/session4_interoperability.sh |

| Time | Endpoint Tested | Response | Reference File |
|---|---|---|---|
| Step 3 | **Test API GET request**<br>/todos/408/tasksof<br>Documented? DOCUMENTED<br>Expected: 200 listing project 207 | **Status 200: OK**<br>Observed: HTTP 200 — {"projects":[{"id":"207","title":"interop project", "completed":"false","active":"false","description":"","tasks":[{"id":"408"}] }]}<br>Result: **As Expected**<br>Side note: Project <-> todo link confirmed two-way | logs/exploratory_session4_interoperability.log exploratory/session4_interoperability.sh |
| Step 4 | **Test API POST request**<br>/todos/408/categories body: {"id":"224"}<br>Documented? DOCUMENTED<br>Expected: 201: link category 224 to todo 408 | **Status 201: Created**<br>Observed: HTTP 201 — (empty response body)<br>Result: **As Expected**<br>Side note: Empty response body | logs/exploratory_session4_interoperability.log exploratory/session4_interoperability.sh |
| Step 5 | **Test API GET request**<br>/categories/224/todos<br>Documented? DOCUMENTED<br>Expected: Todo 408 should be listed for category 224 | **Status 200: OK**<br>Observed: HTTP 200 — {"todos":[]}<br>Result: **Bug**<br>Side note: BUG-01: empty list — link stored one way only | logs/exploratory_session4_interoperability.log exploratory/session4_interoperability.sh |
| Step 6 | **Test API POST request**<br>/todos/408/categories body: <category><id>224</id></category><br>Documented? DOCUMENTED<br>Expected: 201: XML payloads are supported for all operations | **Status 404: Not Found**<br>Observed: HTTP 404 — {"errorMessages":["Could not find thing matching value for id"]}<br>Result: **Bug**<br>Side note: BUG-12: 404 "could not find thing matching value for id"<br>Format: XML request; log evidence | logs/exploratory_session4_interoperability.log exploratory/session4_interoperability.sh |
| Step 7 | **Test API DELETE request**<br>/categories/224<br>Documented? DOCUMENTED<br>Expected: 200: category deleted | **Status 200: OK**<br>Observed: HTTP 200 — (empty response body)<br>Result: **As Expected** | logs/exploratory_session4_interoperability.log exploratory/session4_interoperability.sh |
| Step 8 | **Test API GET request**<br>/todos/408/categories<br>Documented? DOCUMENTED<br>Expected: Empty list: the category no longer exists | **Status 200: OK**<br>Observed: HTTP 200 — {"categories":[{"id":"224","title":"interop category","description":""}]}<br>Result: **Bug**<br>Side note: BUG-10: deleted category 224 still listed on the todo, with its title | logs/exploratory_session4_interoperability.log exploratory/session4_interoperability.sh |
| Step 9 | **Test API DELETE request**<br>/todos/408<br>Documented? DOCUMENTED<br>Expected: 200: todo deleted (cleanup) | **Status 200: OK**<br>Observed: HTTP 200 — (empty response body)<br>Result: **As Expected** | logs/exploratory_session4_interoperability.log exploratory/session4_interoperability.sh |
| Step 10 | **Test API DELETE request**<br>/projects/207<br>Documented? DOCUMENTED<br>Expected: 200: project deleted (cleanup) | **Status 200: OK**<br>Observed: HTTP 200 — (empty response body)<br>Result: **As Expected** | logs/exploratory_session4_interoperability.log exploratory/session4_interoperability.sh |
| Close | **Review findings and cross-reference bugs** | Findings, concerns, risks and new test ideas reviewed and recorded in Section 4; bugs entered in bug_reports.md with the same BUG numbers used there. | bug_reports.md |

## Section 3: Referenced Files (Scripts, Programs, Screen Shots, Video Clips, Spreadsheets, etc)

Table 3: Descriptions of Referenced Files and Created Scripts

| File Name | Description |
|---|---|
| exploratory/session4_interoperability.sh | Curl capability script used in this session: demonstrates every capability exercised, in order. Shared helper: exploratory/common.sh. |
| logs/exploratory_session4_interoperability.log | Raw session log: the exact status code and response body of every request in Table 2, in step order. This log is the evidence for the Response column (no screenshots were taken; the log is replayable via the script). |
| session_notes/CH4_Interoperability_Notes.pdf | This session-notes document. The teacher-template data is contained in this PDF itself: participant/charter information in Section 1, the per-request method, method type, documented status, expected behaviour, expected-vs-observed verdict, observed behaviour, side notes and XML/evidence details in Section 2, and the summary of findings, concerns and test ideas in Section 4. |
| bug_reports.md | Consolidated bug form and the full bug reports (summary, description, impact, steps to reproduce) for the bugs in Table 4. |
| tests/test_interoperability_api.py, tests/test_bugs_expected_behavior.py, tests/test_bugs_actual_behavior.py | Unit tests created from this session's findings, including the expected-behaviour / actual-behaviour pairs for each numbered bug. |
| Part_A_Report.pdf | Final Part A report: deliverables, findings, suite structure and repository description. |

## Section 4: Summary of Session Findings

### 4.1 Summary of Session Findings

Capabilities identified:

- A todo can belong to several projects and categories at once; project <-> todo links are two-way — partly documented.
- Deleting a todo or project removes project <-> todo links; deleting objects never cascades to the other side's objects.
- Relationship creation from JSON bodies on all three entity pairs.

What was tried:

Create one todo, project and category; link todo to project and read back from the todo side; link todo to category and read back from the category side; create the same relationship with an XML body; delete the category and re-read the todo; delete the todo and project (cleanup).

What we learned:

- Project <-> todo links are genuinely bidirectional.
- Todo -> category links are one-way: the category side stays empty (BUG-01).
- Creating a relationship with an XML body returns 404 while the same link by JSON returns 201 (BUG-12).
- Deleting a category leaves a ghost link on the todo: the deleted category is still listed, with its title (BUG-10). The mirror case (deleted todo on a category) is BUG-11.
- Changing a todo's doneStatus does not affect its project's completed flag (checked in the unit suite).

Decisions made:

- Always inspect both ends of a link after every delete.
- Create all objects inside the session/tests so default data is never mutated.
- The /shutdown endpoint (swagger only) was deliberately NOT executed: it would stop the service under test.

Table 4: Descriptions of Identified Bugs

| Bug | Description |
|---|---|
| BUG-01 — Todo→category link is not visible from the category side | POST /todos/408/categories {"id":"224"} returns 201, but GET /categories/224/todos returns an empty list. The category side never sees the link. |
| BUG-12 — Relationship creation fails when the id is sent as XML | POST /todos/408/categories with <category><id>224</id></category> returns 404 “Could not find thing matching value for id”, while the identical JSON request returns 201. XML clients cannot create relationships although XML is advertised for all payloads. |
| BUG-10 — Deleting a category leaves a ghost link on the todo | After DELETE /categories/224 returns 200, GET /todos/408/categories still lists category 224 with its title, although GET /categories/224 is 404. Referential integrity is broken and deleted data remains exposed. |
| BUG-11 — Mirror ghost link (confirmed in follow-up unit tests) | Deleting a todo likewise leaves it listed on its category (GET /categories/{id}/todos). Found by turning this session’s ghost-link probe into the paired bug tests in the unit suite. |

### 4.2 Concerns Identified in Session / Areas of Potential Risk

- BUG-01, BUG-10, BUG-11, BUG-12.
- Referential integrity is not enforced for category links: ghost links expose deleted data (privacy) and break clients that dereference them.
- XML clients cannot create relationships at all, although XML is advertised for all payloads.

### 4.3 New Testing Ideas Identified in Session

- Delete then re-create an object with the same title and re-check links.
- Link cycles (todo <-> project <-> category).
- Stress with many links per object.
- Repeat the ghost-link probes for project <-> todo links in XML.